

공시 Agent 기술 제안서

제10회 2026 미래에셋증권 AI Festival · 공시 Agent 과제 · 팀 냥냥

1. 제안 요약

본 시스템은 LLM의 역할을 문장 생성으로 한정한다. 문서 검색, 정정 이력에서의 최신 값 선택, 금액 계산, 생성된 답변의 검증은 모두 파이썬 코드가 담당한다. HyperCLOVA X(이하 HCX)는 질문을 실행 계획으로 바꾸는 첫 호출과 정리된 근거를 문장으로 만드는 마지막 호출, 질문당 두 번만 사용한다.

이 원칙은 개발 과정에서 겪은 실패를 바탕으로 정했다. 자체 평가셋 58문 만점에 도달하기까지 31회의 실험을 기록했고, 프롬프트 지시로 해결되지 않던 문제가 코드 검증으로 해결되는 사례가 반복해서 확인됐다. 근거에 없는 날짜가 답변에 섞이는 문제, 정정 전의 금액을 인용하는 문제, API 한도 초과 시 답변 품질이 저하되는 문제를 모두 코드 장치로 차단했다. 본 제안서는 각 장치가 필요했던 이유와 동작 방식을 실제 사례와 함께 설명한다.

2. 문제 정의

과제는 DART 공시 4,204건을 근거로 질문에 답하는 API 서버의 구축이다. 공시 데이터를 직접 다루면 일반 문서 검색에는 없는 세 가지 문제가 확인된다.

첫 번째 문제는 정정 공시다. 같은 사건의 공시가 원본과 여러 정정보정으로 갈라지며, 최초 공시 1건에 정정 14건이 이어진 사례도 있다. 코퍼스에서 확인한 현대건설 사우디 아미랄 플랜트 계약의 경우 최초 계약금액 6조 5,545억 원이 최종 정정에서 3조 901억 원으로 변경됐다. 원본만 검색하면 두 배 이상 차이 나는 값을 정답처럼 제시하게 된다. 정정 문서의 본문 구조도 오답의 원인이 된다. 문서 앞부분이 정정 전과 정정 후 값을 나란히 적은 대조표여서, 읽는 위치가 틀리면 폐기된 값을 가져온다.

두 번째 문제는 LLM의 수집·산술 오류다. 근거 6건을 주고 정리를 요청했을 때 2건만 사용한 사례, 3건의 합계를 요청했을 때 후보 7건 중 무관한 3건을 골라 더한 사례가 개발 중에 나왔다. 요구한 항목 수를 채우지 못한 답변은 정답 문자열이 포함돼 있어도 근거 완전성 기준을 충족하지 못한다. 이 유형의 오류는 프롬프트 수정으로 빈도가 줄었을 뿐 사라지지 않았다.

세 번째 문제는 운영 제약이다. HCX 호출에는 분당 한도가 있어 평가 트래픽이 몰리면 429 응답이 발생한다. 부하 테스트에서 확인된 위험은 서버 중단이 아니라, 생성 실패 시 검색 결과를 나열한 답변이 정상 응답과 같은 형식으로 반환된다는 점이었다. HTTP 200에 응답 필드도 모두 갖추므로 외형으로는 구분되지 않는다. 서버 자원 또한 4GB 메모리 안에서 임베딩 모델과 벡터 인덱스를 모두 수용해야 한다.

이 세 문제에 대한 대응을 다음 장에서 순서대로 설명한다.

3. 제안 방법

3.1 계획, 실행, 합성의 3단 파이프라인

질문이 들어오면 HCX에 실행 계획만 먼저 요청한다. 플래너 호출의 출력은 JSON 형식의 단계 목록이고, 각 단계는 코드로 구현된 다섯 개 도구 중 하나를 지정한다. 공시 제목으로 필터링하는 collect, 본문 벡터 검색인 find, 원공시를 지정해 후속 정정·철회·해지를 계보로 추적하는 trace, 연도별 재무 표를 파싱하는 yeartab, 합계와 증감률을 계산하는 compute다. 계획이 확정되면 코드가 단계를 순서대로 실행해 근거를 수집하고, 마지막에 HCX를 한 번 더 호출해 수집된 근거를 문장으로 만든다.

"삼성SDI 유상증자의 예정 발행가와 확정 발행가를 비교해 달라"는 질문이면 원본 공시를 찾는 단계와 최신 정정본을 찾는 단계가 계획에 함께 실리고, 두 결과를 비교 모드로 합성한다. 이전 구조에서는 질문마다 유형 번호 하나를 골라 단일 처리 경로만 사용했는데, 복합 질문에서 처리되지 않은 절반을 LLM이 지어내는 환각이 발생했다. 계획 구조에서는 단계 조합으로 같은 질문이 처리된다. 단순 질문이면 단계가 하나여서 이전과 동일하게 동작하며, HCX 호출 횟수는 어느 경우든 질문당 두 번으로 같다.

3.2 계획에 대한 코드 교정

플래너 역시 LLM이므로 오류를 낸다. 개발 중 확인된 오류 유형은 여섯 가지이며, 모두 코드가 계획을 받아 교정한 뒤 실행한다. 질문에 "2023년 4월"이라고만 있는데 계획에 4월 1일이라는 구체 날짜가 생성되는 경우가 대표적이다. 코드는 질문 원문에서 연월일 패턴을 정규식으로 추출해 두고, 계획의 날짜가 그 집합에 없으면 날짜 조건을 제거하고 월 단위 조건으로 낮춘다. 존재하지 않는 날짜로 검색이 좁혀져 정답 문서를 놓치는 사고를 막기 위한 조치다. 반대로 계획에 누락된 단계를 추가하는 교정도 있다. 질문에 "이후 해지되었는가" 같은 후속 추적 표현이 있는데 계보 추적 단계가 없으면 코드가 단계를 추가한다. 단계를 제거하는 교정은 검색 범위를 줄여 위험하므로, 추가하는 방향으로만 개입한다.

프롬프트에 지시문을 추가하는 방식은 두 차례 시도했으나 모두 실패했다. 지시문 한 줄이 무관한 문항 두 개를 오답으로 만든 사례를 확인한 뒤로는, 발동 조건을 코드로 좁힐 수 있는 교정만 적용한다.

3.3 정정 이력의 그래프 관리

코퍼스 전처리 단계에서 공시 4,204건의 정정 관계를 그래프로 구축한다. 어느 접수번호가 어느 접수번호로 대체되었는지 기록한 correction_map이다. 검색 도구들은 이 그래프를 공유하며 기본값으로 최신 정정본을 선택한다. 질문이 원본 값을 요구하면 계획의 version 축이 original로 지정돼 정정 전 문서를 가져오고, 이력 전체를 물으면 체인 전체를 싣는다. 정정 체인이 중간에 두 갈래로 갈라지는 사례도 코퍼스에 존재하므로, 회수가 한쪽 갈래를 놓치는 경우에 대비해 분기를 포함한 전체 계보를 접수번호 각주로 답변에 표시한다.

정정 문서 본문의 대조표는 읽는 위치를 코드로 고정해 대응했다. 라벨 뒤에 구분자만 사이에 두고 금액이 연달아 나열된 형태면 마지막 값이 정정 후 값이다. 또한 정정공시는 문서 앞부분이 대조표로

채워져 실제 내역이 뒤쪽에 오므로, 본문을 읽는 분량 제한을 정정 문서에 한해 확대했다. 정답 금액이 문서의 4,210자 지점에 있어 기본 제한에서는 잘려 나가던 사례가 이 조치로 해결됐다.

3.4 생성된 답변에 대한 재검사

합성이 끝난 답변은 합성 직후와 공통 출구 두 곳에서 검증을 거친다. 질문이 금액을 물었는데 답변에서 누락됐으면 근거 원문에서 해당 항목의 값을 인용해 보충하며, 이때 답변이 실제로 인용한 문서의 값만 사용한다. 인용하지 않은 문서의 숫자를 근거 원문으로 표기하는 것 자체가 결함임을 자체 고난도 평가에서 확인했기 때문이다. 공통 출구에서는 답변의 날짜가 근거에 존재하는지 검증해 없으면 추적 기록에 경고를 남기고, 답변 문장이 근거에 없는 표현으로 작성됐는지 귀속 검사를 수행한다. 코퍼스 밖 자료를 참고하라는 권유나 공시에 없는 평가성 수식이 생성되면 해당 문장만 제거한다. 마지막으로 회수한 근거 공시를 접수번호와 함께 답변 하단에 표시해, 모든 답변에 근거 공시를 표시하라는 요건을 코드 수준에서 보증한다.

출력 잘림 문제에서는 검증 장치 자체의 결함이 드러났다. 다건 나열 답변이 글자 중간에서 끊긴 채 통과한 것을 발견하고 원인을 추적한 결과, 잘림 경고 코드가 API 응답에 존재하지 않는 필드명을 참조하고 있었다. 경고는 한 번도 발동한 적이 없는 코드였다. 실제 응답 스키마를 확인 호출로 확정해 필드명을 수정하고, 나열 건수가 많은 질문은 생성 분량 제한을 건수에 비례해 늘리되 응답 지연 상한 안에서 제한했다.

3.5 장애 상황의 응답 유지

호출 한도 초과는 평가 기간 중 반드시 발생한다고 가정하고 설계했다. 호출 단위로는 단계적으로 길어지는 백오프 재시도와 60초 상한을 두고, 요청 단위로는 90초 예산을 모든 호출이 공유해 한 문항의 지연이 뒤 문항으로 번지지 않게 한다. 계획 호출이 실패하면 코드가 질문을 직접 해석해 계획을 구성하는 경로로 전환하고, 합성 호출까지 실패하면 이미 수집된 근거를 코드가 정리해 반환한다. 이 최종 폴백 답변에는 일시적인 제약으로 완결된 답변을 생성하지 못했다는 고지를 답변 앞에 표시한다. 검색 결과 나열이 정상 답변과 같은 형식으로 나가는 것보다, 한계를 밝히고 근거를 제공하는 쪽이 평가 취지에 맞는 대응이라고 판단했다.

프롬프트 공격은 LLM 호출 전에 코드 판정으로 처리한다. 시스템 프롬프트 유출 요구와 투자 조언 유도는 호출 없이 거절하고, 거짓 전제가 포함된 질문은 전제만 거절한 뒤 공시에 실제로 기재된 값을 함께 제시해 근거 완전성을 유지한다. 오탐이 미탐보다 손실이 크다는 원칙에 따라 패턴을 좁게 유지하며, 공시 용어와 겹치는 표현은 판정 목록에서 제외해 정상 질문이 거절되지 않도록 했다. 질문이 코퍼스 수록 범위 밖이면 답변을 보류하되, 수록된 기간과 기업 정보를 근거로 함께 제시해 답변할 수 없는 이유를 증명한다.

4. 시스템 구성

전체 시스템은 NCP 가상 서버 1대(2vCPU, 메모리 4GB)에서 동작하며, 구성은 이 자원 제약을 중심으로 설계했다.

NCP 서버 (2vCPU / 4GB, Docker)

```
+-----+
| FastAPI  GET /answer
|      |
| 공시 Agent 파이프라인
| (가드 판정, 플래너, 도구 5종, 검증 체인)
|      |
| 검색 계층 (전 모듈 공유 싱글턴)
|   - 임베딩 모델 bf16 로딩
|   - FAISS 인덱스 SQ8 압축
|   - 청크 메타 SQLite 지연 조회
+-----+
|
| HCX API              전처리 산출물: 인덱스, correction_map
(질문당 2회 호출)   제공 데이터: 공시 원문, 문서 대장(manifest)
```

메모리 사용량의 대부분은 검색 계층에서 발생한다. 절감 조치는 세 가지다. 임베딩 모델을 bfloat16으로 로딩해 용량을 절반으로 줄였고, FAISS 인덱스는 스칼라 양자화로 압축했으며, 청크 메타데이터는 메모리에 올리지 않고 SQLite에서 필요할 때 조회한다. 세 조치 없이 기동했을 때 메모리 사용량은 11.3GB로 서버 용량의 세 배 가까이 됐다. 이 때문에 파이프라인 코드는 서버가 생성해 둔 검색기 인스턴스를 재사용하며, 절감 조치를 우회하는 자체 로딩이 발생하면 기동 로그에 경고가 남는다. 부하 테스트 중 메모리 사용량은 약 2GB로 안정적으로 유지됐다. 동일 구성을 다른 환경(Windows)에서 기동해 재확인했을 때도 2.9GB로 4GB 한도 안이었다.

전처리는 배포 전에 1회 수행한다. 공시 4,204건을 청크로 분할해 임베딩하고, 정정 관계를 correction_map 그래프로 구축한다. 기업 별칭과 종목코드는 제공된 문서 대장(manifest)의 정식 명칭으로 코드가 정규화한다. 런타임에는 이 산출물만 읽으므로 요청 처리 중에 코퍼스 원문 전체를 탐색하지 않는다.

5. 기능 흐름

질문 하나의 처리 과정을 실패 분기까지 포함해 정리하면 다음과 같다.

질문

```
|
|-- 공격 패턴? --- 예 --> 호출 0회 거절 (거짓 전제면 공시 기재값 동봉)
|
플래너 호출 (HCX 1차) ----- 429/실패 --> 코드가 계획 수립
|
계획 교정 (기업명 정규화, 지어낸 날짜 제거,
           빠진 추적 단계 추가)
|
도구 실행 (collect / find / trace / yeartab / compute)
|
|-- 수집 0건? --- 예 --> 수록 범위 근거를 첨부한 기권
|
근거 조립 (정정 문서는 읽기 분량 확대)
|
합성 호출 (HCX 2차) ----- 429/실패 --> 코드가 근거 정리
|                                     + 한계 고지 서두
검증과 보강 (날짜 검증, 귀속 검사, 금액 보충,
           잘림 감지, 위험 표현 제거)
|
근거 공시 표시 (접수번호 트레일러, 정정 계보 각주)
|
응답 {question_id, question, answer,
      retrieved_context, think_trace}
```

어느 분기로 진행되어도 HTTP 200과 다섯 필드를 갖춘 응답이 반환된다. think_trace에는 계획 JSON부터 단계별 필터링 건수, 코드 계산 내역, 검증 경고까지 기록되므로 심사자가 추론 과정을 재검증할 수 있다. 실제 기록 한 건을 발췌하면 다음과 같다. 후보 50건이 세 단계 필터로 1건까지 좁혀지는 과정이 그대로 남는다.

```
[플래너] 계획 1단계 통과 -> 계획: {"steps": [{"tool": "collect",
  "corp": "SK하이닉스", "year": 2024, "month": 4,
  "topic": "신규 시설투자"}], "answer": {"mode": "single"}}
-> s1: SK하이닉스 50건 / 연도 2024 필터 14건
   / 키워드(시설투자/신규) 1건 / 최신본 1건
-> 조 단위 금액 한글 표기 병기 -> 합성 완료
```

폴백이 발동한 요청도 어느 지점에서 전환됐는지 기록이 남아, 운영 중 품질 저하를 사후에 집계할 수 있다.

6. 사용자 시나리오

자체 평가셋 58문을 배포 서버에서 순차 실행했을 때 응답 시간은 중앙값 8.1초, 상위 90%가 15.7초 안이었다. 아래 세 사례는 배포 서버에 실제로 들어온 질문과 저장된 응답에서 선정했으며, 답변 원문은 발췌다.

정정 공시를 추적한 사례. "엘에스일렉트릭이 2024년에 공시한 초고압 변압기 시설 증설 투자의 투자금액은 얼마인가"에 시스템은 기재정정 공시를 근거로 선택해 "증액 205억원이 결정되어 설비투자 금액이 기존 803억원에서 총 1,008억원으로 확정되었습니다"라는 원문을 제시했다. 정정 그래프 없이 원본만 검색했다면 803억 원을 답했을 질문이다.

거짓 전제를 포함한 질문 사례. 공시에 없는 매출 급증을 사실처럼 전제하고 배경 설명을 요구하는 질문에 "질문에 전제로 제시된 내용은 제공된 공시에서 확인되지 않습니다"라고 전제를 거절한 뒤, 해당 기업 반기보고서와 사업보고서의 실제 기재 내용을 접수번호와 함께 제시했다. 거절만 하고 근거를 제시하지 않으면 근거 완전성 기준에서 불리해지므로 이 형식으로 정했다.

수록 범위 밖 질문 사례. "삼성전자의 2021년 연결기준 매출액은 얼마인가"에는 2021년이 제공 데이터 범위 밖임을 밝히고, 코퍼스에 수록된 해당 기업 공시의 시작 시점을 근거 공시와 함께 제시했다.

7. 기대효과와 확장성

7.1 평가 관점에서의 설계 대응

시스템의 설계 결정은 평가지표에 하나씩 대응하도록 구성했다.

평가지표	대응 설계
정확성	정정 그래프의 최신본 선택(3.3), 산술의 코드 전담(3.1), 대조표 위치 고정(3.3)
근거 완전성	계획 기반 다단계 수집(3.1), 빠진 단계를 보태는 교정(3.2), 거짓 전제에도 기재값 동봉(3.5)
요구사항 충족	나열 건수 비례 생성 분량과 잘림 감지(3.4)
근거 기반	날짜 검증과 귀속 검사(3.4), 인용 문서의 값만 보충하는 규칙(3.4)
추론 논리성	계획 JSON과 단계별 기록을 담은 think_trace(5)
안전성 및 신뢰성	호출 전 코드 가드(3.5), 위험 표현 문장 제거(3.4)
정보한계 대응	수록 범위를 증명하는 기권(3.5), 폴백 시 한계 고지(3.5)
근거 공시 표시	회수 문서 접수번호 전수 수록과 정정 계보 각주(3.4)

7.2 한계와 남은 과제

자체 평가셋 58문 만점이 모든 결함의 부재를 의미하지는 않는다. 난도를 높여 별도 제작한 고난도 10문 세트에서는 5문을 통과했으며, 실패 원인은 분석을 마치고 문서로 관리하고 있다. 플래너 계획을 LLM이 생성하므로 같은 질문에도 실행 간 계획이 달라질 수 있다는 점도 확인했다. 계획 품질이 나쁘게 나온 실행에서도 답변이 정상 답변의 형식을 흉내 내지 않도록, 폴백 경로 전체에 한계 고지를 강제해 두었다.

가장 큰 미해결 과제는 교차 기업 질문이다. "A사가 B사에 발주한 계약"을 물으면 해당 공시는 B사 이름으로 접수돼 있는데, 현재 검색은 질문의 주어인 A사만 탐색한다. 본문의 계약 상대 필드로 공시 주체를 넘나드는 검색 수단이 필요하며, 검색 구조의 변경이므로 제출 안정성을 위해 개선 과제로 남겼다. 15개 문서에 이르는 긴 정정 체인에서 회수가 상위 일부에서 끊기는 문제, 검색은 성공했으나 질문이 요구한 항목이 근거에 없을 때 한계를 밝히지 못하는 문제도 같은 이유로 후순위에 두었다. 세 항목 모두 원인 분석이 끝나 있어 개선 방법도 정리돼 있다.

7.3 확장 방향

계획과 도구를 분리한 구조에서 확장 지점은 도구 목록이다. 새로운 공시 유형이나 질의 패턴이 나타나면 도구를 추가하고 플래너에 그 존재를 알리면 되고, 기존 경로는 변경되지 않는다. 정정 계보 그래프는 공시가 매일 추가되는 실서비스 환경에도 적용할 수 있다. 신규 공시를 그래프에 연결하는 증분 갱신으로 충분하다. 검증 체인은 생성 모델과 독립적이므로 모델을 교체해도 안전장치가 유지된다.